```
//========================================== var , let , const
============================================================//

//---------------------------------------- Var ----------------------------------//

var : Global scoped or function scoped

//---------- Globle Scope ----------------//

var a = 10
function f(){
   console.log(a)  // 10
}
f();
console.log(a); // 10

//---------- function scope ----------------//

function f() {

   // It can be accessible any where within this function
   var a = 10;
   console.log(a)  // 10
}
f();
console.log(a); // a is not defined

//---------- Re-declare and Update ----------------//

var a = 10
var a = 8  // Re-declare
a = 7   // Update

console.log(a)  // 7

//---------- Access before declare ----------------//

console.log(a); // undefined
var a = 10;

//---------------------------------------- Let ----------------------------------//

let : block Scope
```

```
//---------------------- Block Scope ------------------------//

let a = 10;
function f() {
    let b = 9
    console.log(b);  // 9
    console.log(a);  // 10
}
f();

//---------------------- Function block----------------------//


let a = 10;
function f() {
    if (true) {
            let b = 9
            console.log(b); // 9
    }

    console.log(b); // It gives error as it defined in if block
}
f()


console.log(a) // 10

//-------------------- Re-declare and Update------------------//

let a = 10

// It is not allowed
let a = 10

// It is allowed
a = 10

//----------------Re-declare out side block -------------------//

let a = 10
if (true) {
    let a=9
```

```
    console.log(a) // It prints 9
}
console.log(a) // It prints 10

//---------- Access before declare ----------------//

console.log(a); // Cannot access 'a' before initialization
let a = 10;


//---------------------------------------- Const ----------------------------------//

const : block Scope

const a = 10;
function f() {
    a = 9 // We are changing the value of the const variable so that it returns an error.
    console.log(a)
}
f();


const a = {
    prop1: 10,
    prop2: 9
}

// It is allowed
a.prop1 = 3

// It is not allowed
a = {
    b: 10,
    prop2: 9
}



//-------------------------------------- Call , Bind , Apply ------------------------------------------------//

var person = {firstName: 'Jon', lastName: 'deni'};
function say(greeting) {
        console.log(greeting + ' ' + this.firstName + ' ' + this.lastName);
```

```
}

say.call(person, 'Hello'); // Hello Jon deni
say.apply(person, ['Hello']); // Hello Jon deni
var sayHelloJon = say.bind(person,"hello"); // Hello Jon deni
sayHelloJon();


//------------------------------------------ Currying Using Recursion----------------------------------------//
var Sum = 0;
function sum(a){
  Sum += a;
  return function ff(b){
          if(b===".")
          {
          return Sum;
          }
          else
          {

          return sum(b)
          }
  }
}

let sum = a => b=> b == "."  ? a : sum ( a+b );
let a = sum(2)(3)(5)(3)(4)(1)(".");
console.log("Sum = ",a);

//------------------------------------------ Closure ------------------------------------------------//

function Parent(){
          let i = 10;
          function inner(){
          console.log(i);
          }
          i++;
          return inner();
}
Parent();

//------------------------------ Destructuring & (Rest & Spread Oprator)
-------------------------------------------//
```

```
let Arr = [22,45,68,78];
let [firstValue, secondValue, ...restArr] = [...Arr,501];

let userDetails = {name: "nikunj",Age: 23};
let {name,...restObject} = {...userDetails,"lastname":"nikunj"};

let ArrayOfObjects = [{name: "nikunj",Age: 23},{name: "rahul",Age: 25}]
let [firstObject,...restArrayObject] = [...ArrayOfObjects,{name: "manish",Age: 28}];

//-------------------------------- (Rest & Spread Oprator) ------------------------------------------//


function myBio(firstName, lastName, ...otherInfo) {
  return otherInfo;
}

myBio("Oluwatobi", "Sofela", "CodeSweetly", "Web Developer", "Male");

function myBio(firstName, lastName, company) {
  return `${firstName} ${lastName} runs ${company}`;
}

myBio(...["Oluwatobi", "Sofela", "CodeSweetly"]);

var abc = ['a', 'b', 'c'];
var def = ['d', 'e', 'f'];
var alpha = [ ...abc, ...def ];

//---------------------------------- Reduce Function ------------------------------------------------//


function sum(...theArgs) {
  return theArgs.reduce((previous, current) => {
          return previous + current;
  });
}

sum(1, 2, 3);

const occurrences = "abcdefccklak".split("").reduce(function (acc,curr) {
  return acc[curr] ? ++acc[curr] : (acc[curr] = 1), acc;
},{});
```

```
//---------------------------------- Sort Function ----------------------------------------------------//

const cars = [
  {type:"Volvo", year:2016},
  {type:"Saab", year:2001},
  {type:"BMW", year:2010}
];

cars.sort(function(a, b){return a.year - b.year});

//---------------------------------- Type Of ----------------------------------------------------------//

typeof "John"              // Returns "string"
typeof 3.14                // Returns "number"
typeof NaN                 // Returns "number"
typeof false               // Returns "boolean"
typeof [1,2,3,4]           // Returns "array"
typeof {name:'John', age:34}  // Returns "object"
typeof new Date()                  // Returns "object"
typeof function () {}      // Returns "function"
typeof myCar                       // Returns "undefined" *
typeof null                // Returns "object"

//------------------------------ SetTimeOut ----------------------------------------------------------//

for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1);
}

for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 1);
}
```

Answer 3 3 3 and 0 1 2 **Explanation — **Because of the event queue in JavaScript, the setTimeout callback function is called after the loop has been executed. Since the variable i in the first loop was declared using the var keyword, this value was global. During the loop, we incremented the value of i by 1 each time, using the unary operator ++. By the time the setTimeout callback function was invoked, i was equal to 3 in the first example.

In the second loop, the variable i was declared using the let keyword: variables declared with

the let (and const) keyword are block-scoped (a block is anything between { }). During each iteration, i will have a new value, and each value is scoped inside the loop.

to avoid

```
for (var i = 0; i < 5; i++) {
   (function(x) {
         setTimeout(function() { console.log(x); }, x * 1000 );
   })(i);
}
```

```
//---------------------------- Deep and shallow Copy ---------------------------------------------//
```

Use the spread (...) syntax
Use the Object.assign() method
Use the JSON.stringify() and JSON.parse() methods
lodash librery _.cloneDeep()

```
const person = {
         firstName: 'John',
         lastName: 'Doe'
};


// using spread ...
let p1 = {
         ...person
};

// using  Object.assign() method
let p2 = Object.assign({}, person);

// using JSON
let p3 = JSON.parse(JSON.stringify(person));

//------------------------------- Flat Arr ----------------------------------------------------//

const arr = [1, 2, 3, 4, [5, 6, 7, [8]]];
         function flatArr(arr){
         let new_arr = [];
```

```
        arr.forEach(element => {
        if(Array.isArray(element)){
        new_arr.push(...flatArr(element));
        }else{
        new_arr.push(element);
        }
        });
        return new_arr;

        }
})

console.log([].concat.apply([], arr))

 //----------------------------- Promise channing ------------------------------------------------//

    Promise.resolve(1)
    .then((x) =>{ console.log("first"); throw new Error('My Error')})
    .catch(() => { console.log("first 1"); return 1})
    .then((x) => { console.log("second"); throw new Error('My Error') })
    .catch((x) => { console.log("second 1"); return 2})
    .then(() => { console.log("third"); throw new Error('My Error')})
    .catch((x) => { console.log("third 1"); return 3 })


 //-------------------------- callback Hell ------------------------------------------------//


    function createOrder(cart,function(){
          function payment(function(){
                function wallet(function(){
                  })
          })
    })

//--------------------------- Duplicate data remove from array ----------------------------------------//

    var arr = ["apple", "mango", "apple",
        "orange", "mango", "mango"];

        function removeDuplicates(arr) {
        return arr.filter((item,
        index) => arr.indexOf(item) === index);
```

```
        }

    function removeDuplicates(arr) {
        return [...new Set(arr)];
        }

//--------------------------- Practical Javascript ------------------------------------------//

    (function(){
      var a = b = 3;
    })();
    b = 3;
    var a = b;
    console.log(typeof a) // undefined
    console.log(b) // 3

    ---------------------------------------------------------------------------

    var myObject = {
        foo: "bar",
        func: function() {
        var self = this;
        console.log("outer func:  this.foo = " + this.foo);
        console.log("outer func:  self.foo = " + self.foo);
        (function() {
        console.log("inner func:  this.foo = " + this.foo);
        console.log("inner func:  self.foo = " + self.foo);
        }());
        }
    };
    myObject.func();

    outer func:  this.foo = bar
    outer func:  self.foo = bar
    inner func:  this.foo = undefined
    inner func:  self.foo = bar

    ---------------------------------------------------------------------------

    function foo1()
    {
      return {
            bar: "hello"
```

```
  };
}

function foo2()
{
  return
  {
        bar: "hello"
  };
}

console.log(foo1());  // console.log(0.1 + 0.2);
console.log(foo2());  // undefined

--------------------------------------------------------------------------------

console.log(0.1 + 0.2); // 0.30000000000000004
      console.log(0.1 + 0.2 == 0.3); // false
console.log(1 +  "2" + "2"); //  "122"
console.log(1 +  +"2" + "2"); // "32"
console.log(1 +  -"1" + "2");  // "02"
console.log(+"1" +  "1" + "2"); // "112"
console.log( "A" - "B" + "2");  // "NaN2"
console.log( "A" - "B" + 2); // NaN
console.log("0 || 1 = "+(0 || 1));  // 1
console.log("1 || 2 = "+(1 || 2));  // 1
console.log("0 && 1 = "+(0 && 1));  // 0
console.log("1 && 2 = "+(1 && 2));  // 2

console.log(1 < 2 < 3); // true  left to right
      console.log(3 > 2 > 1); // false

--------------------------------------------------------------------------------

var arr1 = "john".split('');  ["j","o","h","n"]
var arr2 = arr1.reverse(); ["n","h","o","j"]
var arr3 = "jones".split('');
arr2.push(arr3); ["n","h","o","j",["j","o","h","n","s"]]
console.log("array 1: length=" + arr1.length + " last=" + arr1.slice(-1));
console.log("array 2: length=" + arr2.length + " last=" + arr2.slice(-1));

array 1: length=5 last=j,o,n,e,s
      array 2: length=5 last=j,o,n,e,s
```

```
--------------------------------------------------------------------------------

var a={},b={key:'b'},c={key:'c'};
a[b]=123;
a[c]=456;
console.log(a[b]); // 456

--------------------------------------------------------------------------------

var x = 21;
var girl = function () {
        console.log(x);
        var x = 20;
};
girl (); // undefined


/---------------------------------- Higher order function ------------------------------//

let arr = [1, 2, 3, 4, 5];

function map(arr, mapFunc) {
        const mapArr = []; // empty array
        // loop though array
        for(let i=0;i<arr.length;i++) {
                const result = mapFunc(arr[i]);
                mapArr.push(result);
                }
                return mapArr;
}
const mapFunc = num => num * 2;
const squareArr2 = map(arr, mapFunc);
console.log(squareArr2); // prints [1, 4, 9, 16, 25]


function filter(arr, filterFunc) {
        const filterArr = []; // empty array
        // loop though array
        for(let i=0;i<arr.length;i++) {
                const result = filterFunc(arr[i], i, arr);
                // push the current element if result is true
                if(result)
```

```
                filterArr.push(arr[i]);
                }
                return filterArr;

}
const filterFunc = num => num % 2 === 0
const oddArr2 = filter(arr, filterFunc);
console.log(oddArr2); //prints [2, 4]


function reduce(arr, reducer, initialValue) {
        let accumulator = initialValue === undefined ? 0 : initialValue
        // loop though array
        for(let i=0;i<arr.length;i++)
        accumulator = reducer(accumulator, arr[i], i, arr);
        return accumulator;
}
const sumReducer = (accumulator, currentValue) => accumulator + currentValue;
const sum2 = reduce(arr, sumReducer, 5);
console.log(sum2); //prints 15


console.log(Math.floor(Math.random() * 10) + 1)
      txt.charAt(0).toUpperCase()+ text.slice(1)
```